

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

2. Background & Business Objective	3
3. Product Purpose	5
4. System Functionalities	5
5. User Stories & Use Cases	9
6. Features Included (Scope Definition)	11
7. Features Not Included (Scope Control)	12
8. Assumptions & Constraints	12
9. Risks & Questions Throughout Development	14

Project Name: MediSync+

Version: 1.0

1. Project Overview

- **Problem Statement:** When a patient is in the ward, they receive constant professional care. However, the transition to home is often fragmented. Patients struggle to understand complex discharge notes, forget medication schedules, and fail to track symptoms and risks accurately. On the hospital side, post-discharge documentation and follow-ups are manual, unstructured, and labor-intensive.
- **Target Domain:** The focus is on enabling a continuous, automated workflow that extends the "clinical environment" into the patient's home.
- **Proposed Solution Summary:** MediSync+ is an AI-native, multi-agent orchestrator mobile application that acts as a "virtual nurse" that deals with post-discharge workflow. It integrates fragmented hospital databases (prescriptions, ward history, doctor notes) to automate medication reminders, checklist management, and symptom monitoring, ensuring a continuous workflow of care from hospital to home.

2. Background & Business Objective

- **Background of the Problem:** Currently, discharge summaries and follow-up instructions are provided as static, often confusing documents.
 - **Patients Side:** The transition from clinical to home care often leaves **patients** without the structured guidance they received in the ward, leading to a sense of uncertainty during recovery. Patients frequently struggle to interpret complex medical notes and adhere to medication schedules, which can compromise their

safety. Furthermore, the absence of continuous professional monitoring makes it difficult to track vital health metrics or identify high-risk symptoms that require immediate intervention

- **Hospital Side:** Medical staff must manually call patients for follow-ups, and data collected post-discharge rarely makes it back to the hospital database in a structured way.
- **Importance of Solving This Issue:** To transform the post-discharge process from a fragmented hand-off into a continuous workflow of care. This ensures that:
 - **Continuous Guidance:** Patients are never left "lost" after discharge, as the virtual nurse provides the similar level of structured support found in a hospital ward.
 - **Adherence & Clarity:** Complex prescriptions and doctor reminders are translated into simple, automated schedules, significantly reducing the risk of medication errors.
 - **Proactive Safety:** Continuous monitoring and risk detection agents act as a safety net, identifying dangerous symptoms early and alerting medical staff before a condition worsens.
 - **Equitable Access:** Vulnerable groups, including the elderly and those with limited financial resources, receive high-quality recovery support that would otherwise be unavailable outside of a hospital setting.
- **Strategic Fit / Impact:** MediSync+ aligns with the shift toward AI-driven telehealth, bridging the gap between clinical intervention and home-based recovery through real-time data synchronization.

3. Product Purpose

3.1. Main Goal of the System: To provide an intelligent, automated bridge between hospital treatment and home recovery using multi-agent orchestration.

3.2. Intended Users (Target Audience):

- **Primary:** Discharged patients (Elderly, Athletes, Cancer/Chronic disease patients and low-income patients who cannot afford prolonged hospital stays).
- **Secondary:** Hospital Staff (Doctors and Nurses monitoring recovery).

4. System Functionalities

4.1. Description: The system acts as a multi-agent orchestrator that serves as a virtual nurse for patients post-discharge. It bridges the gap between hospital-grade care and home recovery by integrating fragmented medical data into a continuous, automated workflow. Using an intelligent "Brain" (Orchestrator Agent), the application processes historical ward data, prescriptions, and doctor notes to manage the patient's recovery journey through natural interactions and automated monitoring.

4.2. Key Functionalities:

- **Medicine Management:** Automated scheduling, reminders, and "next dose" tracking. Push notifications will be sent for reminders.
- **Follow-up Coordination:** Synced with hospital databases to remind users of upcoming visits. Push notifications will be sent for reminders.
- **Today's Checklist:** A daily task list with push notifications for incomplete tasks. For some specific tasks (e.g., "Check Blood Pressure"), agents will record the data into the follow-up database as filled in by the users.

- **Condition Follow-up:** Records vitals (Weight, Blood Pressure, Blood Sugar) in the follow-up database and analyzes symptoms.
- **Interactive Chatbot:** Answers questions about medicine and health advice based on medical knowledge and patient-specific databases.
- **Monitor and Alert:** For each feature mentioned above, agents will keep monitoring and detecting if abnormalities or risk is detected from users input, agents will explicitly take action.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The system utilizes Z AI GLM as the core LLM to power the multi-agent orchestrator. This model was selected specifically for its advanced capabilities in handling the complex medical logic required for post-discharge care:

- **Reasoning Ability:** The model excels at clinical reasoning, allowing it to analyze unstructured data from "Historical Ward Databases" and "Doctor/Nurse advice" to identify potential health risks. It can distinguish between general symptoms and dangerous symptoms that require immediate hospital escalation.
- **Planning & Orchestration:** As an Orchestrator Agent, Z AI GLM effectively decomposes high-level goals—such as "Manage Patient Discharge"—into actionable sub-tasks for specialized agents, including the Medicine Agent, Follow-up Agent, and Risk Trajectory Agent.
- **Tool-Use & Integration:** The model serves as the "Brain" that interfaces with the MediSync Action layer. It demonstrates high proficiency in tool-calling to record data to the "Hospital Database," trigger "Notification reminders," and generate "Today's Checklist" based on prescription data.

4.3.2. Prompting Strategy: The MediSync+ team employs a **multi-step agentic prompting** approach. This strategy is selected for several critical reasons:

- **Workflow Continuity:** Post-discharge recovery is not a single event but a "continuous workflow". Multi-step prompting allows the Orchestrator Agent to maintain state across different recovery phases, such as moving from medication reminders to symptom logging.
- **Decomposition of Complex Tasks:** Medical care requires precision; by using a multi-agentic approach, the system can break down a broad goal like "Discharge Care" into specific sub-tasks handled by specialized agents for medicine, follow-up, and risk detection.
- **Contextual Accuracy:** This strategy ensures that the **Orchestrator** can first parse the patient's intent and then retrieve relevant data from the "Historical Ward Database" or "Prescription Database" before generating a response, minimizing the risk of medical inaccuracy.
- **Dynamic UI Generation:** Agentic prompting facilitates the "Prompt-to-UI" flow, where the model identifies required components based on the user's specific disease and renders the appropriate record-filling windows or schedules.

4.3.3. Context & Input Handling: MediSync+ enforces a two-tier input size policy managed by InputProcessor.kt. For normal inputs, the system accepts up to 2,000 characters (MAX_NORMAL_SIZE) per user turn; a noise-reduction pass using preprocess() removes repeated symbols and normalises whitespace before the text reaches the Z AI GLM API, reducing token consumption. For oversized inputs between 2,001 and 8,000 characters, the system silently truncates the input to the 4,096-token boundary and prepends an in-chat notice:

“Your message was too long and has been shortened. Please ask a specific question.” Inputs exceeding 8,000 characters are truncated locally and never forwarded to the API. Conversation history is context-pruned to the last 10 messages (ChatAgent.kt) to prevent token bloat in long sessions. For structured agent tasks (MedicineAgent, SymptomAgent), raw patient data is pre-extracted from the AgentToolRegistry into targeted JSON payloads rather than forwarding full medical histories, ensuring per-request token budgets remain within the 4,096-token limit per turn and approximately 6,000 tokens total including the system prompt and tool manifest.

4.3.4. Fallback & Failure Behavior: When the Z AI GLM API returns an off-topic, malformed, or hallucinated response, MediSync+ applies a structured three-tier fallback chain: (1) Validation Layer – ResponseValidator.kt inspects every LLM output before it is acted upon. It checks that any medication names in the response exist in the patient’s active MedicationEntity list and that required JSON fields are present for structured agents. If validation fails, the response is discarded and the fallback chain is triggered immediately rather than retrying an expensive second LLM call. (2) Local Heuristic Fallback – SymptomAgent and MedicineAgent each implement a fallbackAnalysis() method that uses local keyword matching against a curated set of CHF red-flag terms (e.g., “breathless”, “swollen”, “chest pain”) to determine a risk level (CRITICAL / WARNING / STABLE) without any LLM call. This ensures patient safety monitoring continues even during a Z AI GLM API outage. (3) Graceful Error State – If all upstream layers fail, ChatAgent returns a safe, hardcoded human-escalation message: “I’m here for you. For urgent concerns, please call your care team at Ward 4B or dial 999.” Critical medication alerts that cannot be confirmed via LLM are escalated directly to the clinician dashboard using the rule-based AgentToolRegistry send_clinician_alert tool, ensuring no patient safety event is silently dropped. There is no automatic LLM retry to prevent runaway API costs; all retries are managed by WorkManager’s exponential backoff policy for background sync tasks only.

5. User Stories & Use Cases

- **User Stories:**

- **As a patient**, I want a virtual nurse to guide me through my recovery so that I don't feel lost after leaving the hospital.
- **As an elderly patient**, I want automated reminders for my medications so that I never forget a dose due to my forgetfulness.
- **As a medical provider**, I want structured data on patient recovery so that we can intervene if dangerous symptoms occur.

- **Use Cases:**

Scenario: The Post-Discharge Recovery Journey This use case describes a typical day for a patient using MediSync+ to manage their health after being discharged from the ward.

- **Medication & Schedule Management:**

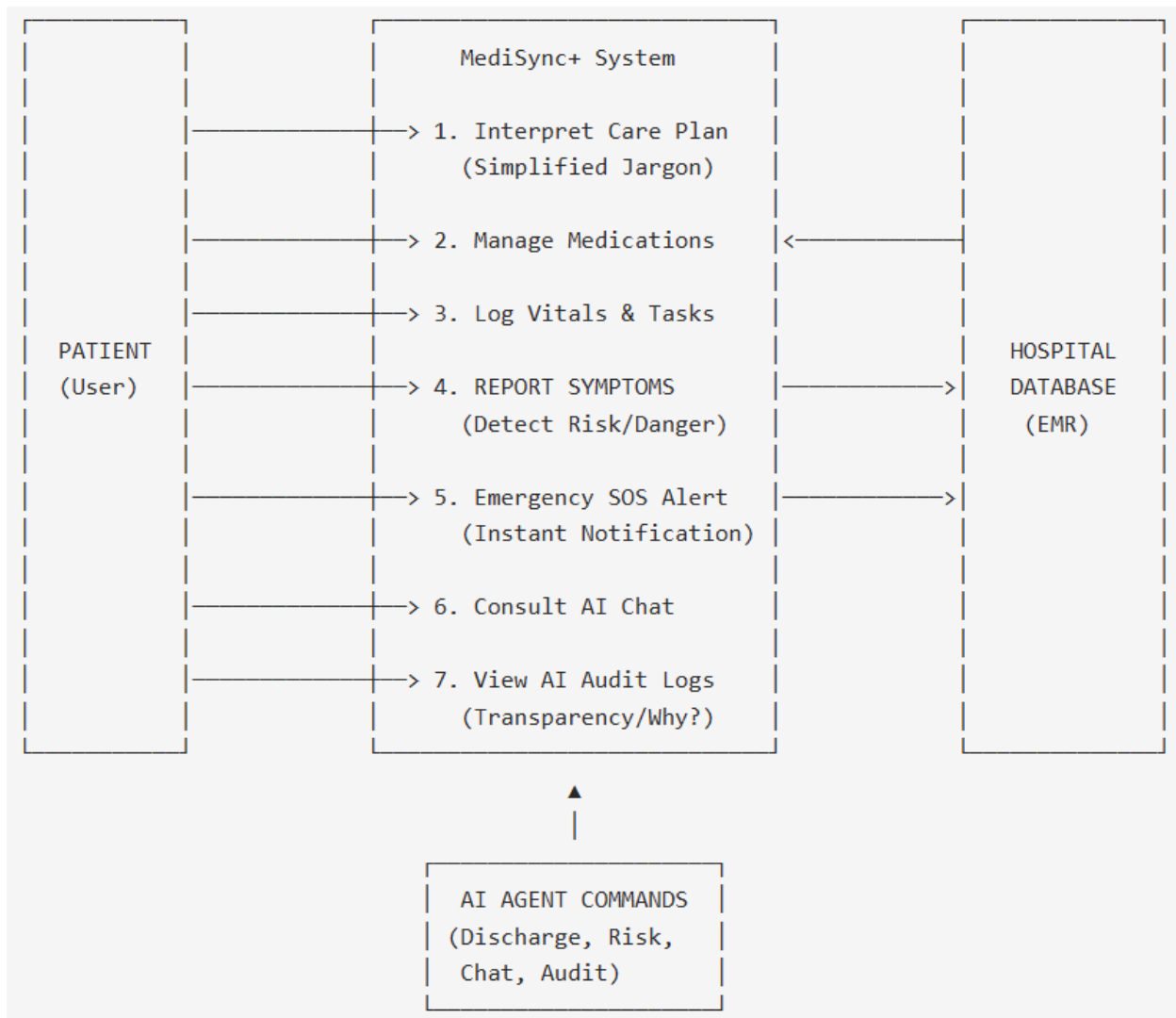
- The Medicine Agent retrieves prescription data from the hospital database.
- It generates a UI design displaying the "Next medicine" and the full daily schedule.
- The user receives a notification reminder when it is time for a dose.

- **Daily Checklist & Vitals Logging:**

- The **Checklist Agent** presents "Today's Checklist," which includes tasks like "Record Blood Pressure".
- If a task is not marked as done, the system sends a follow-up notification.
- Once the user marks a task as complete, a window pops out for the user to fill in records such as weight, blood sugar, or blood pressure based on their specific disease.

- **Symptom Monitoring & Risk Detection:**

- The user describes current feelings or symptoms via the **Chatbot** or the **Condition Follow-up** section.
- The **Risk Trajectory Agent** analyzes these descriptions alongside the patient's historical ward data.
- If the agent identifies **dangerous symptoms**, it immediately triggers a user alert and records the data to the hospital's patient follow-up database.
- **Follow-up Visit Coordination:**
 - The **Follow-up Agent** checks the database for follow-up dates.
 - It automatically populates the user's schedule and sends reminders to ensure the patient does not miss their post-discharge appointment.
- **Interactive Health Consultation:**
 - The user asks the chatbot: "Can I take this medicine with food?".
 - The **Chat Agent** provides an answer based on the integrated medical database and general knowledge.
 - If the chat interaction reveals risky symptoms, the agent automatically updates the hospital database for doctor review.
- **Use Case Diagram:**



6. Features Included (Scope Definition)

- **Multi-Agent Ecosystem:** Includes Orchestrator, Checklist, Chat, Discharge, Follow-up, Medicine, Risk Trajectory, and Symptoms agents.
- **Database Integration:** Bi-directional sync between the app and hospital records.

- **Automated Reminders:** Notification to remind users about medication, upcoming follow-up visit at hospital and incomplete daily tasks.
- **Emergency Alerting:** Real-time risk analysis based on users historical database and symptoms described.

7. Features Not Included (Scope Control)

- Voice, image and video input are not supported.
- Only support Android systems.

8. Assumptions & Constraints

- **LLM Cost Constraint:**

Token Cost Estimation (Average User Session)

- An average session (Checking vitals, logging 1-2 symptoms, and a brief chat interaction) is estimated to consume between 2,500 – 4,500 tokens.
- Morning Check-in: ~1,200 tokens (System prompt + tool calls for medications/vitals + summary generation).
- Symptom Analysis: ~800 tokens (Single turn with tool execution).
- Chat Interaction: ~500–1,500 tokens (Depending on conversation history length).

Design Decisions for Cost Management

A. Aggressive Input Preprocessing & Truncation ([InputProcessor.kt](#))

- Instead of sending raw, potentially bloated user text to the LLM, we use a local utility to "clean" the input before it ever hits the API:
- Noise Reduction: We use preprocess() to remove repeated symbols and normalize spacing, reducing the raw character count (and thus token count).

- Hard Constraints: We enforce a MAX_NORMAL_SIZE (2000 chars). If a user inputs more than 8000 chars, the system truncates the input locally rather than passing an expensive, oversized prompt to the LLM.
- Early Signal Extraction: For simple queries, we can extract "signals" locally (like symptom keywords) to provide context, allowing us to use shorter, more targeted prompts.

B. Local Logic Fallbacks ([SymptomAgent.kt](#))

- We avoid calling the LLM for every single interaction if validation fails or if the system is under heavy load:
- Heuristic Fallback: If the LLM returns an invalid or nonsensical response, the SymptomAgent switches to a fallbackAnalysis method. This uses local keyword matching to determine risk levels (CRITICAL/WARNING) without a second expensive LLM retry if the first one was unusable.

C. Context Pruning ([ChatAgent.kt](#))

- In ChatAgent.respond(), we specifically limit the conversation history:
- Kotlin// From [ChatAgent.kt](#)
- Val historytext = conversationHistory.takeLast(10).joinToString("\n")
- By only taking the last 10 messages, we prevent "token bloat" where long-running conversations become exponentially more expensive as the session progresses.

D. Validation Before Retry ([AgentOrchestratorHelper.kt](#))

- The system validates the "relevance" and "structure" of the AI output locally using the ResponseValidator. This ensures we don't pay for tool execution or subsequent rounds if the initial response is hallucinated or off-topic, failing fast rather than continuing an expensive, multi-round tool-calling chain.
- **Technical Constraints:** Requires seamless API access to hospital Electronic Health Records (EHR).
- **Performance Constraints:** Z AI GLM API responses for symptom analysis must return within 3 seconds under normal load (single session). Medication notification delivery

must occur within 10 seconds of the scheduled alarm trigger time. LLM API concurrent load target: average response time < 800ms for up to 30 simultaneous sessions; p95 must not exceed 1,200ms. All critical clinician alerts must be escalated to the hospital dashboard within 10 seconds of detection. On-device Room DB read/write operations must complete within 200ms to maintain UI responsiveness.

- **User Input:** Effectiveness relies on the user (or caregiver) accurately marking tasks and describing patients' daily condition.
- **Privacy:** All data must be handled according to medical data protection regulations (e.g., HIPAA/GDPR equivalent).

9. Risks & Questions Throughout Development

- **Data Integrity:** How do we ensure the Orchestrator doesn't hallucinate important information such as dosage instructions? This is mitigated by the AgentToolRegistry grounding strategy: all agents must retrieve medication data via tool calls before generating clinical responses, and ResponseValidator.kt cross-checks all drug names against the patient's active MedicationEntity list before the response is shown.
- **Z AI GLM API Outage:** What happens if the core LLM becomes unavailable during a patient's care session? Resolved via local heuristic fallback in SymptomAgent.fallbackAnalysis() for risk detection and a hardcoded safe escalation message in ChatAgent, ensuring no care session fails silently. Critical medication alerts are dispatched rule-based via AgentToolRegistry regardless of LLM availability.
- **Notification Delivery Failure:** How do we ensure medication notifications are delivered reliably on Android devices with aggressive battery optimisation (Doze mode)? Addressed by using AlarmManager with high-priority notification channels for medication reminders, combined with WorkManager for checklist and follow-up reminders. Devices with battery optimisation must be prompted to whitelist the app

during onboarding. This is identified as a Critical (16/25) risk in the QA Risk Assessment.

- **Patient Data Privacy:** How do we ensure sensitive medical data is not exposed through LLM prompt leakage or insecure API transmission? All patient data sent to the Z AI GLM API is scoped strictly to the current patient (hardcoded patientId in AgentToolRegistry) with no cross-patient data access possible. API keys are injected via BuildConfig and are never committed to source control. All hospital EHR sync traffic uses HTTPS. Adherence to PDPA (Malaysia) and HIPAA/GDPR-equivalent standards is a mandatory constraint for production deployment.